

Sección 11: Observabilidad Total en Microservicios con LGTM Stack y OpenTelemetry

➔ En spring 4 ya viene integrado OpenTelemetry.

- El problema de la caja negra: Análisis de por qué un sistema distribuido sin observabilidad es imposible de mantener en producción.
- OpenTelemetry: Integración del estándar de la industria para exportar métricas, trazas y logs desde todos los microservicios de forma unificada.
- Stack Grafana LGTM: Despliegue del stack completo — Loki para logs, Tempo para trazas distribuidas y Prometheus para métricas.
- Correlación de logs con Trace ID: Inyección de identificadores únicos en el patrón de logging para vincular eventos de consola con trazas distribuidas.
- El puente Logback-OpenTelemetry: Implementación del `InstallOpenTelemetryAppender` para redirigir el flujo de logs de consola hacia Loki.
- Análisis visual en Grafana: Monitoreo del sistema bajo carga real cruzando métricas, trazas y logs en un solo panel.

159. OpenTelemetry y la observabilidad moderna

Monitorear vs Observar

La diferencia crítica en sistemas distribuidos

✗ Monitoreo Tradicional

Te dice **QUE** algo falló.

"El servicio de inventario está caído"

"La tasa de error subió al 5%"

Pero no te dice **POR QUÉ**.

✓ Observabilidad

Te permite preguntar **POR QUÉ** falló.

"¿Qué pedido causó el error?"

"¿Qué servicio tardó más?"

Sin cambiar el código.

💡 **Clave:** En microservicios, necesitamos poder hacer preguntas que **no anticipamos** cuando escribimos el código. Esa es la promesa de la observabilidad.

¿Qué es OpenTelemetry?

El estándar de código abierto para observabilidad

OpenTelemetry (OTel) es un estándar de la CNCF que unifica la recolección de datos de observabilidad en un solo protocolo: **OTLP** (OpenTelemetry Protocol).

Antes de OTel, cada herramienta tenía su propio formato: Zipkin usaba uno, Jaeger otro, Prometheus otro. Hoy, con OTLP, escribimos nuestro código UNA VEZ y lo enviamos a cualquier backend compatible.



Métricas

CPU, memoria, requests/s, latencia promedio



Trazas

Seguimiento de peticiones entre servicios



Logs

Eventos estructurados con contexto distribuido

Universal: Con OTLP puedes enviar datos a Grafana, Jaeger, Datadog, New Relic, o cualquier backend compatible. Sin vendor lock-in.

El Punto de Inflexión

Por qué Spring Boot 4 cambia todo

Antes de Spring Boot 4

Integrar OpenTelemetry era una **pesadilla**:

- ✗ Agentes externos de Java (-javaagent:opentelemetry.jar)
- ✗ Librerías de terceros en versión "alfa" o "beta"
- ✗ Configuración manual de exporters y samplers

Con Spring Boot 4

Observabilidad nativa con soporte oficial:

- ✓ Starter oficial: **spring-boot-starter-actuator** + OTLP
- ✓ Autoconfiguración out-of-the-box
- ✓ Spring habla OTLP directamente (sin agentes)

Actuator vs OpenTelemetry

¿Competidores o complementarios?

Spring Actuator

-  **Pull-based:** Prometheus raspa endpoints
-  **Health checks:** /actuator/health
-  **Métricas locales:** JVM, HTTP, DB pools
-  **Ideal para:** Monitoreo de instancia

VS

OpenTelemetry

-  **Push-based:** Envía datos activamente
-  **Trazas distribuidas:** TraceID end-to-end
-  **Contexto completo:** Spans entre servicios
-  **Ideal para:** Debugging distribuido

🏆 El Veredicto

No son competidores — son complementarios.

Usaremos **Actuator** para health checks y métricas locales (Prometheus scraping),
y **OpenTelemetry** para el rastro completo de peticiones distribuidas.

Ejemplo: Cuando un pedido falla, **Actuator** te dice que el servicio está caído, y **OTel** te muestra exactamente qué petición causó el error.

Los 3 Pilares de la Observabilidad

Métricas, Trazas y Logs trabajando juntos

1 Métricas (Metrics)

Qué: Números agregados en el tiempo (latencia promedio, requests/segundo, % de CPU)

Para qué: Detectar *que* hay un problema ("La latencia subió 300%")

Ejemplo: Grafana mostrando gráficos de throughput y errores

2 Trazas (Traces)

Qué: El recorrido de UNA petición entre servicios (Trace ID + Spans)

Para qué: Entender *por qué* hay un problema ("Order Service tardó 5s esperando a Inventory")

Ejemplo: Ver el waterfall completo de un pedido lento

3 Logs (Logs)

Qué: Eventos discretos con contexto (timestamp, nivel, mensaje, TraceID)

Para qué: Detalles específicos ("Stock insuficiente para SKU-123 en Inventory Service")

Ejemplo: Logs correlacionados por TraceID para ver la historia completa

 **El poder de OTel:** Estos 3 pilares están **correlacionados** por TraceID. Un spike en latencia (métrica) te lleva a una traza específica, que te muestra los logs relevantes. Todo conectado.

Cómo Fluye la Observabilidad

De tu app al backend de análisis

El Flujo OTLP

- 1 **Instrumentación:** Tu app de Spring Boot genera métricas, trazas y logs
- 2 **Exportación:** Los datos se envían vía **OTLP** (gRPC o HTTP)
- 3 **Collector (Opcional):** Un intermediario que procesa, filtra y enruta datos
- 4 **Backend:** Grafana Tempo, Jaeger, o cualquier backend compatible los almacena
- 5 **Visualización:** Grafana te muestra gráficos, trases, y logs unidos

💡 **Simplicidad:** En desarrollo, podemos saltar el Collector y enviar directamente a Tempo o Jaeger. En producción, el Collector nos da control fino sobre qué enviar y adónde.

** Grafana: Logs en Locky, la trazas en Tempo y métricas en Prometheus

OpenTelemetry: Resumen

- 🔍 **Observabilidad > Monitoreo:** Permite preguntar "por qué" sin código nuevo
- 🌐 **OTel = Estándar Universal:** Un protocolo (OTLP) para todos los backends
- 🔧 **Spring Boot 4:** Observabilidad nativa sin agentes externos
- 📊 **3 Pilares:** Métricas (qué), Trazas (por qué), Logs (detalles)
- 💡 **Actuator + OTel:** Complementarios, no competidores

Con OpenTelemetry, pasamos de "algo está roto" a "sé exactamente qué pedido, en qué servicio, y por qué falló".

160. OpenTelemetry en: Order Service 1

<https://gist.github.com/gchaldy/67b45a2d87eee6e1b116181fb447032e>

➔ Descargar la imagen del stack de grafana:

```
docker-compose.yml x
1  services:
68  rabbitmq:
72      - "5672:5672" # Puerto para comunicación entre microservicios
73      - "15672:15672" # Puerto para la consola de gestión web
74      environment:
75          RABBITMQ_DEFAULT_USER: guest
76          RABBITMQ_DEFAULT_PASS: guest
77      volumes:
78          - rabbitmq_data:/var/lib/rabbitmq # <--- ¡La persistencia!
79
80  # Stack Grafana para los logs, trazas y métricas
81  grafana-lgtm:
82      image: 'grafana/otel-lgtm:latest'
83      ports:
84          - '3000:3000' # ventana visual, navegador, interfaz de grafana
85          - '4317:4317' # Grpc, puerto en alto rendimiento en producción
86          - '4318:4318' # Http
87      environment:
88          - OTEL_METRIC_EXPORT_INTERVAL=5000 # métricas se envían a grafana cada 5s
89
```

➔order-service, agregar la dependencia:

```
Project v
microservices-ecommerce D:\new_projects\java\mic
> .idea
> api-gateway
> config-data
> config-repo-cache
> config-server
> discovery-server
> inventory-service
> notification-service
v order-service
> .mvn
> src
> target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
> product-service
Arquitectura de Microservicios parte 1.pdf

pom.xml (order-service) x
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/
36  <dependencies> Add Starters...
126      <artifactId>spring-cloud-starter-loadbalancer</artifactId>
127      </dependency>
128      <!-- OAuth2 -->
129      <dependency>
130          <groupId>org.springframework.boot</groupId>
131          <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
132      </dependency>
133      <!-- Stack grafana para las metricas, logs y trazas -->
134      <dependency>
135          <groupId>org.springframework.boot</groupId>
136          <artifactId>spring-boot-starter-opentelemetry</artifactId>
137      </dependency>
138      <dependency>
139          <groupId>io.opentelemetry.instrumentation</groupId>
140          <artifactId>opentelemetry-logback-appender-1.0</artifactId>
141          <version>2.21.0-alpha</version>
142      </dependency>
143  </dependencies>
144  <!-- Agregando configuración para spring cloud -->
```

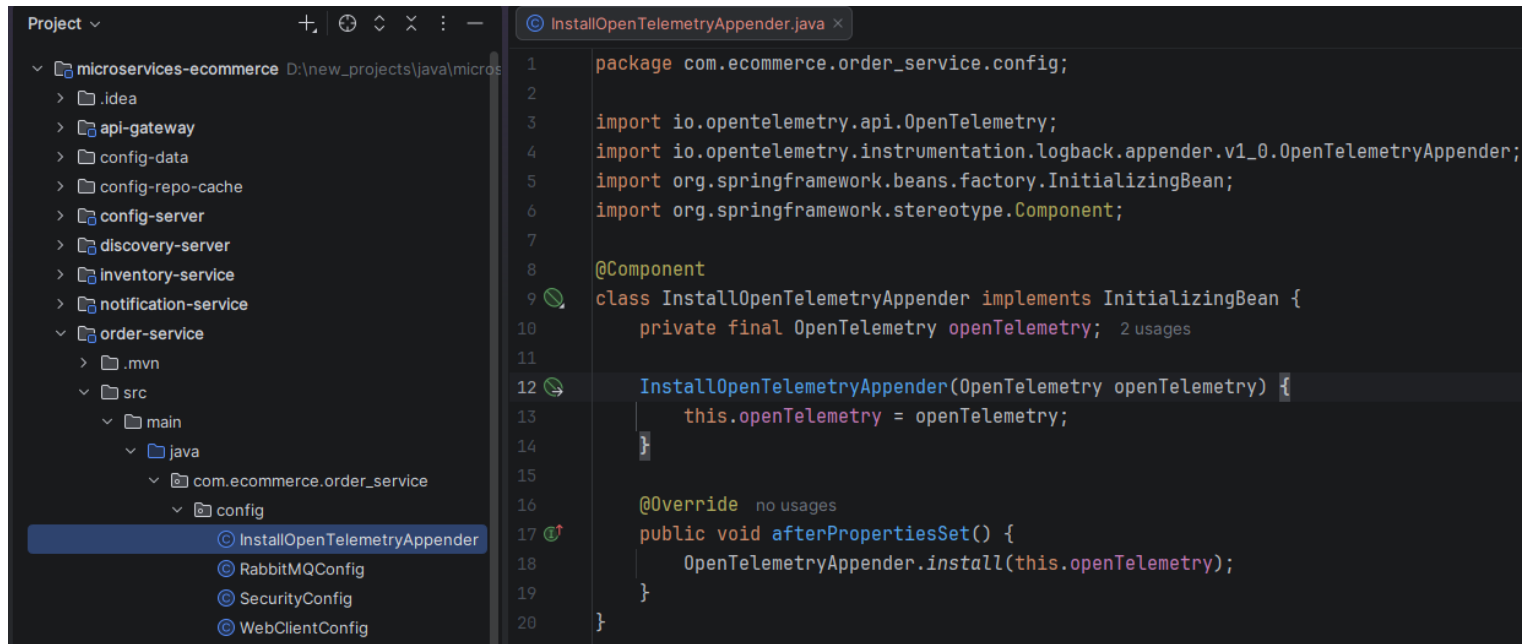
→ En el config-data de order-service:

```
36
37 # ✅ CONFIGURACIÓN UNIFICADA DE MANAGEMENT (Observabilidad + Actuator)
38 management:
39   tracing:
40     sampling:
41       probability: 1.0 # Captura el 100% de las trazas para desarrollo
42     opentelemetry:
43       tracing:
44         export:
45           otlp:
46             endpoint: http://localhost:4318/v1/traces
47       logging:
48         export:
49           otlp:
50             endpoint: http://localhost:4318/v1/logs
51     otlp:
52       metrics:
53         export:
54           url: http://localhost:4318/v1/metrics
55
56   endpoint:
57     health:
58       show-details: always
59
60   health:
61     circuitbreakers:
62       enabled: true
63
64   endpoints:
65     web:
66       exposure:
67         include: "refresh, health, circuitbreakers, prometheus"
68
69 # ✅ CORRELACIÓN DE LOGS (TraceID y SpanID en consola de IntelliJ/Docker)
70 logging:
71   pattern:
72     level: "%5p [%${spring.application.name:},%X{trace_id:-},%X{span_id:-}]"
73
74 # ✅ RESILIENCIA (Circuit Breaker y Retry)
```

161. OpenTelemetry en: Order Service 2

➔ Crear la clase InstallOpenTelemetryAppender en config:

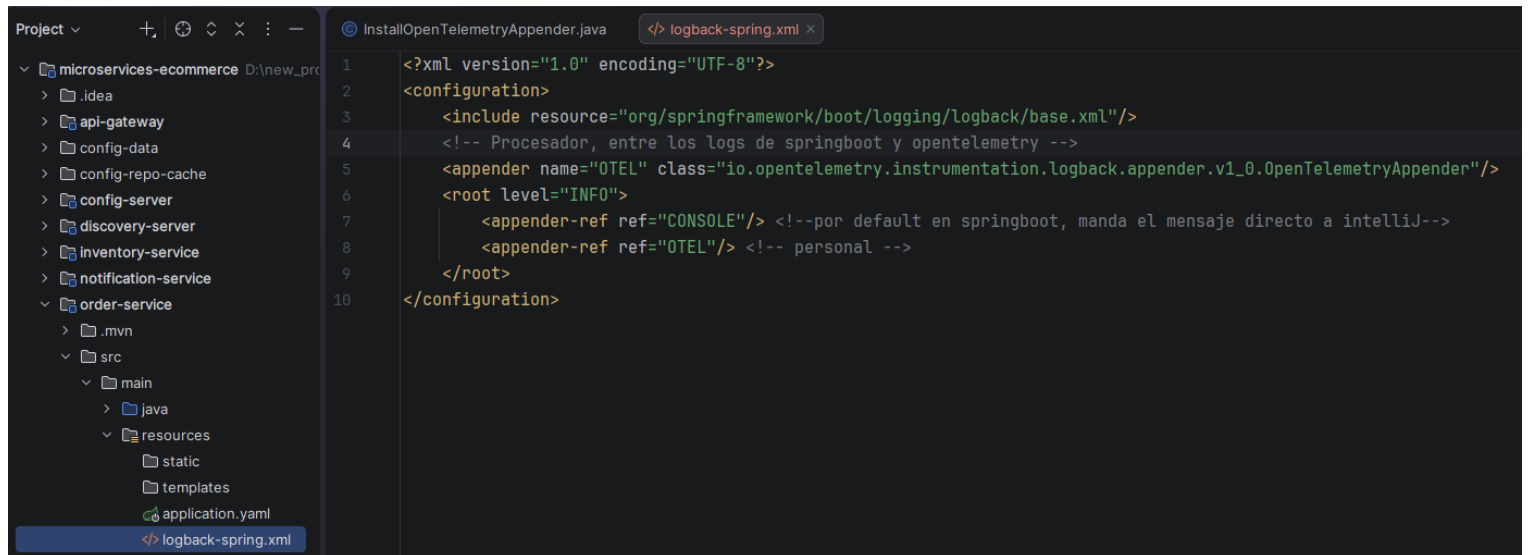
** Puente que conecta toda la información que produce spring boot cuando la api está levantada (peticiones GET, POST, cuando va a la BD, etc) con OpenTelemetry (puente de visibilidad entre Spring Boot y OpenTelemetry)



```
1 package com.ecommerce.order_service.config;
2
3 import io.opentelemetry.api.OpenTelemetry;
4 import io.opentelemetry.instrumentation.logback.appender.v1_0.OpenTelemetryAppender;
5 import org.springframework.beans.factory.InitializingBean;
6 import org.springframework.stereotype.Component;
7
8 @Component
9 class InstallOpenTelemetryAppender implements InitializingBean {
10     private final OpenTelemetry openTelemetry; 2 usages
11
12     InstallOpenTelemetryAppender(OpenTelemetry openTelemetry) {
13         this.openTelemetry = openTelemetry;
14     }
15
16     @Override no usages
17     public void afterPropertiesSet() {
18         OpenTelemetryAppender.install(this.openTelemetry);
19     }
20 }
```

➔ Dentro de resources, crear logback-spring.xml:

Complemento de la clase java

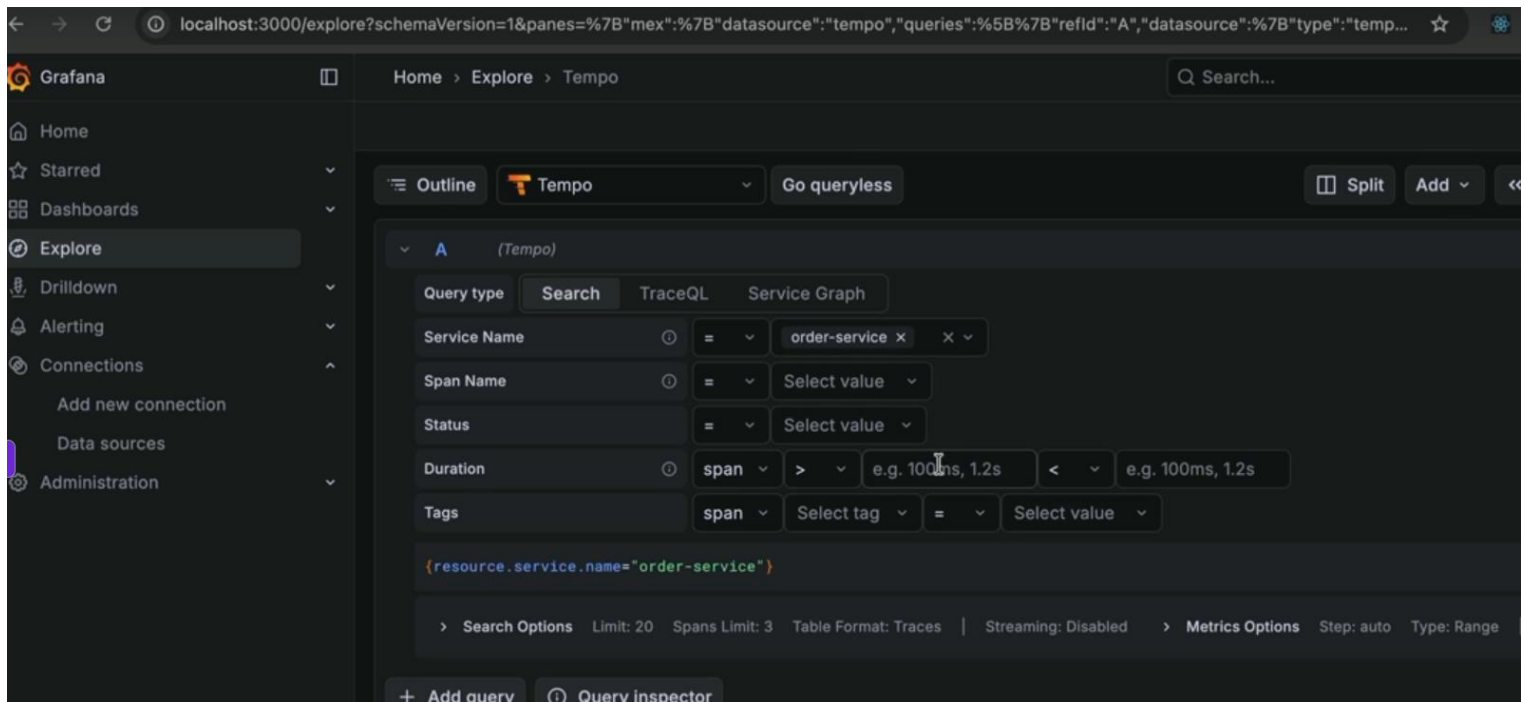


```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <configuration>
3     <include resource="org/springframework/boot/logging/logback/base.xml"/>
4     <!-- Procesador, entre los logs de springboot y opentelemetry -->
5     <appender name="OTEL" class="io.opentelemetry.instrumentation.logback.appender.v1_0.OpenTelemetryAppender"/>
6     <root level="INFO">
7         <appender-ref ref="CONSOLE"/> <!--por default en springboot, manda el mensaje directo a intelliJ-->
8         <appender-ref ref="OTEL"/> <!-- personal -->
9     </root>
10 </configuration>
```


➔ Levantar docker para descargar la imagen:

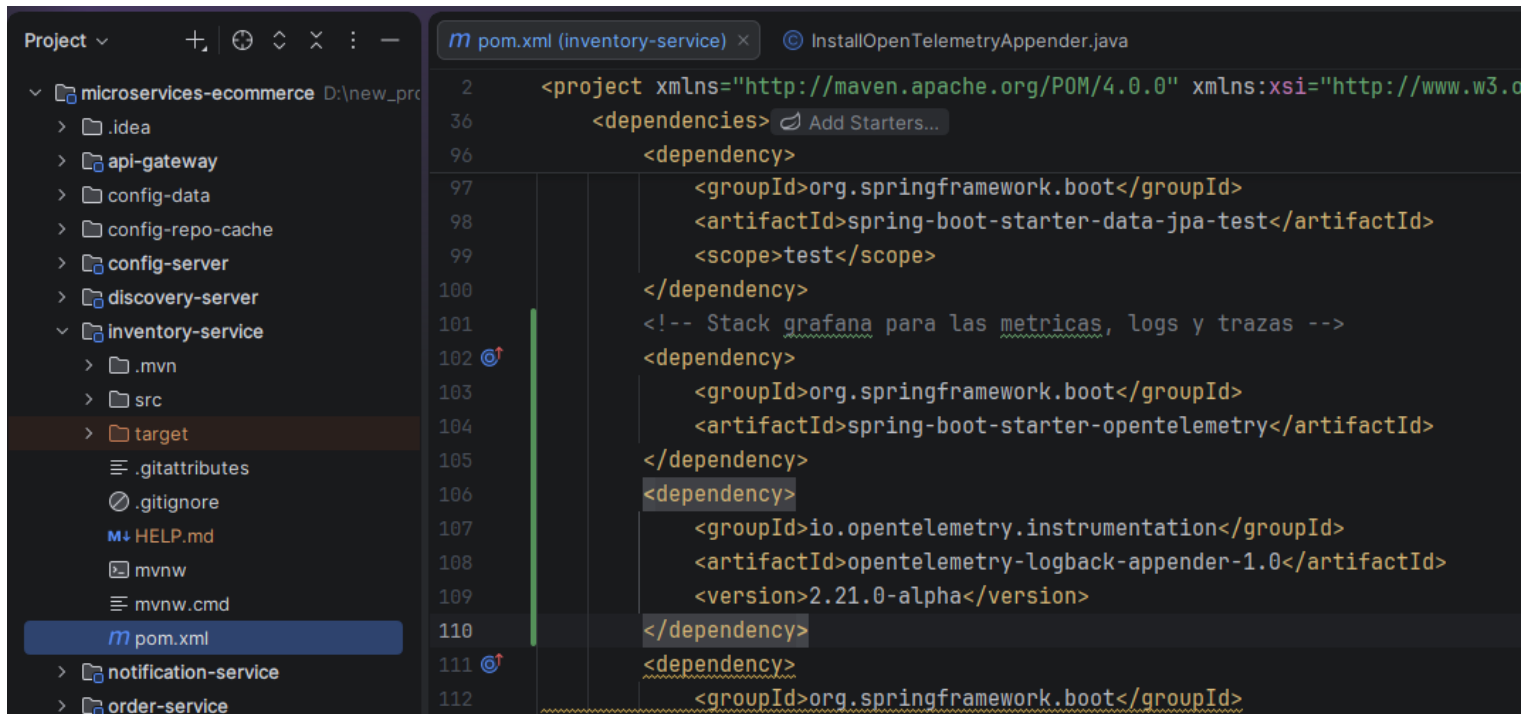
\$> Docker-compose up -d

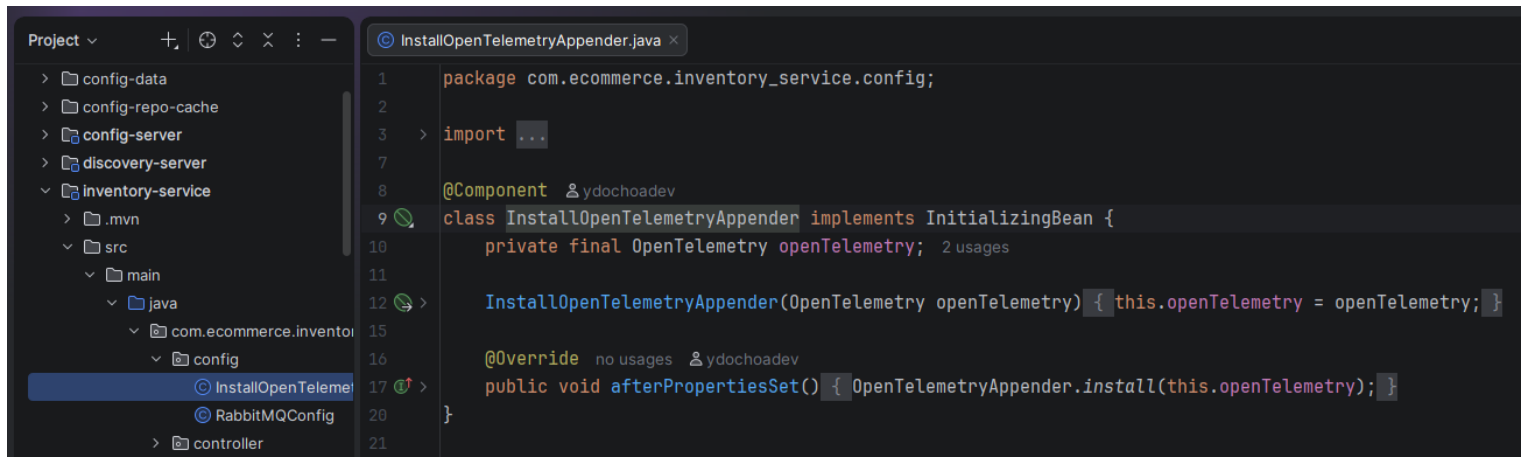
➔ Levantar el proyecto y probar; ir a localhost:3000:



162. OpenTelemetry en: Inventory & Api-gateway

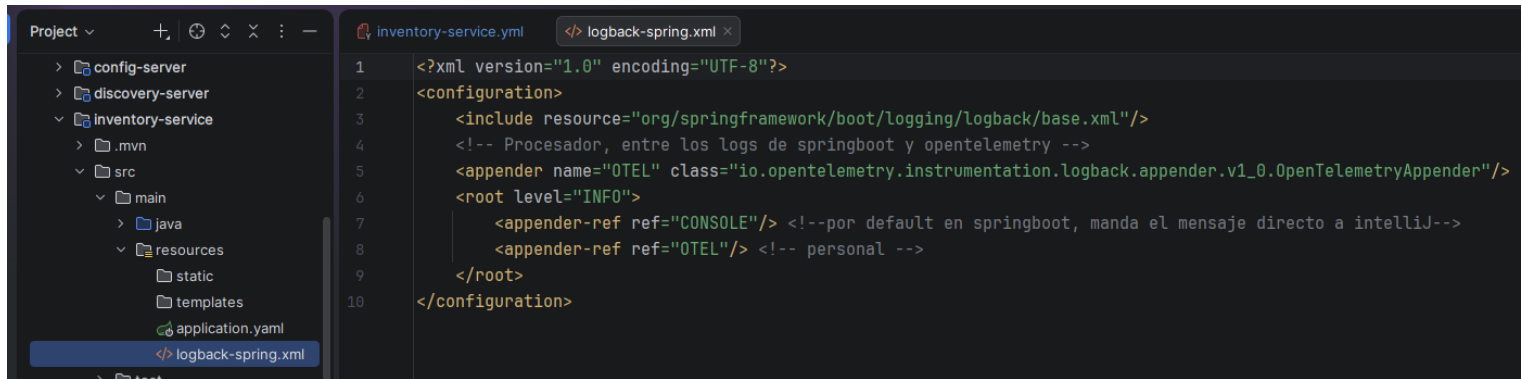
➔ Replicar en inventory:





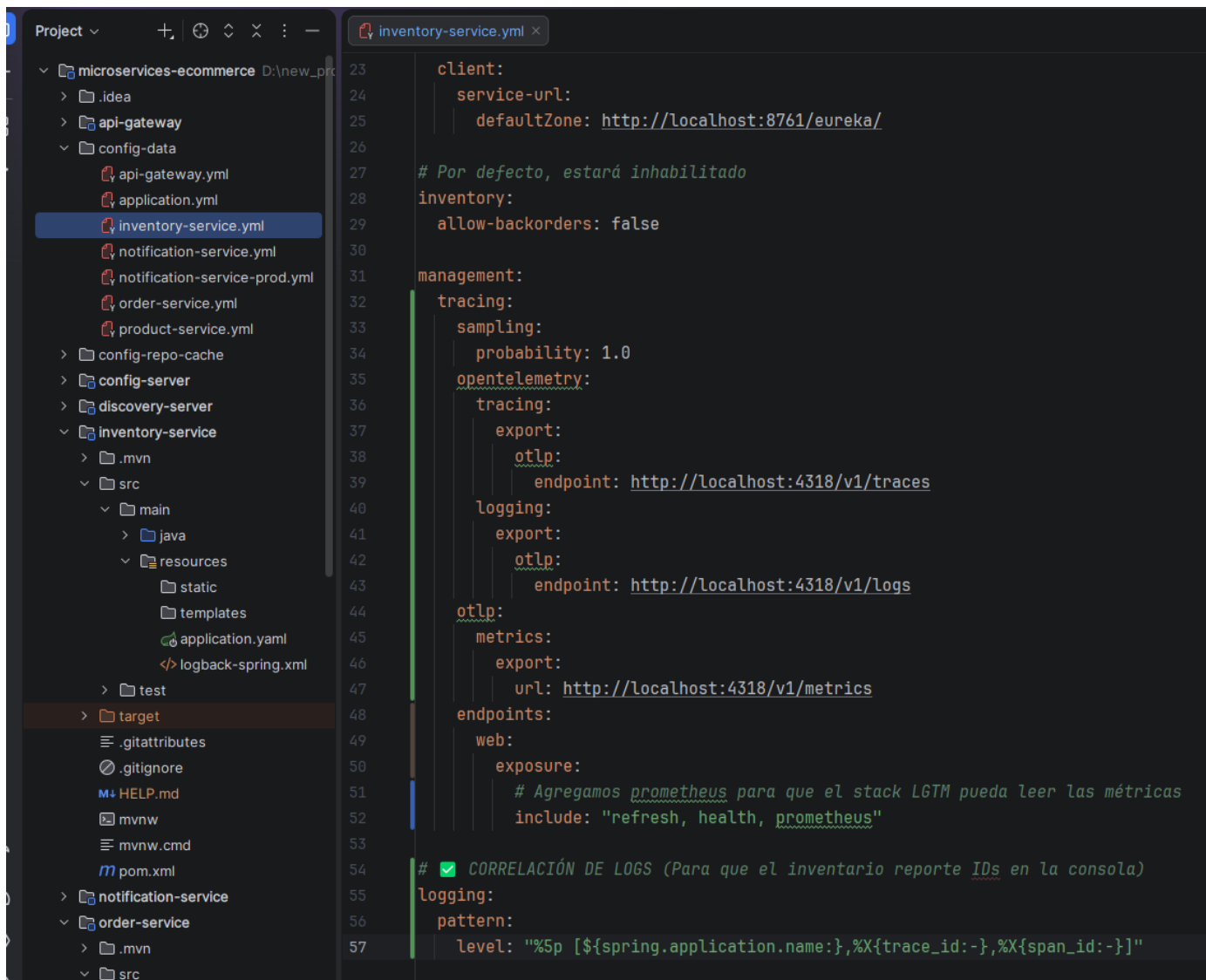
```
1 package com.ecommerce.inventory_service.config;
2
3 > import ...
7
8 @Component &ydochoadev
9 class InstallOpenTelemetryAppender implements InitializingBean {
10     private final OpenTelemetry openTelemetry; 2 usages
11
12 InstallOpenTelemetryAppender(OpenTelemetry openTelemetry) { this.openTelemetry = openTelemetry; }
15
16 @Override no usages &ydochoadev
17 public void afterPropertiesSet() { OpenTelemetryAppender.install(this.openTelemetry); }
20
21 }
```

➔Archivo xml:



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <configuration>
3     <include resource="org/springframework/boot/logging/logback/base.xml"/>
4     <!-- Procesador, entre los logs de springboot y opentelemetry -->
5     <appender name="OTEL" class="io.opentelemetry.instrumentation.logback.appender.v1_0.OpenTelemetryAppender"/>
6     <root level="INFO">
7         <appender-ref ref="CONSOLE"/> <!--por default en springboot, manda el mensaje directo a intelliJ-->
8         <appender-ref ref="OTEL"/> <!-- personal -->
9     </root>
10 </configuration>
```

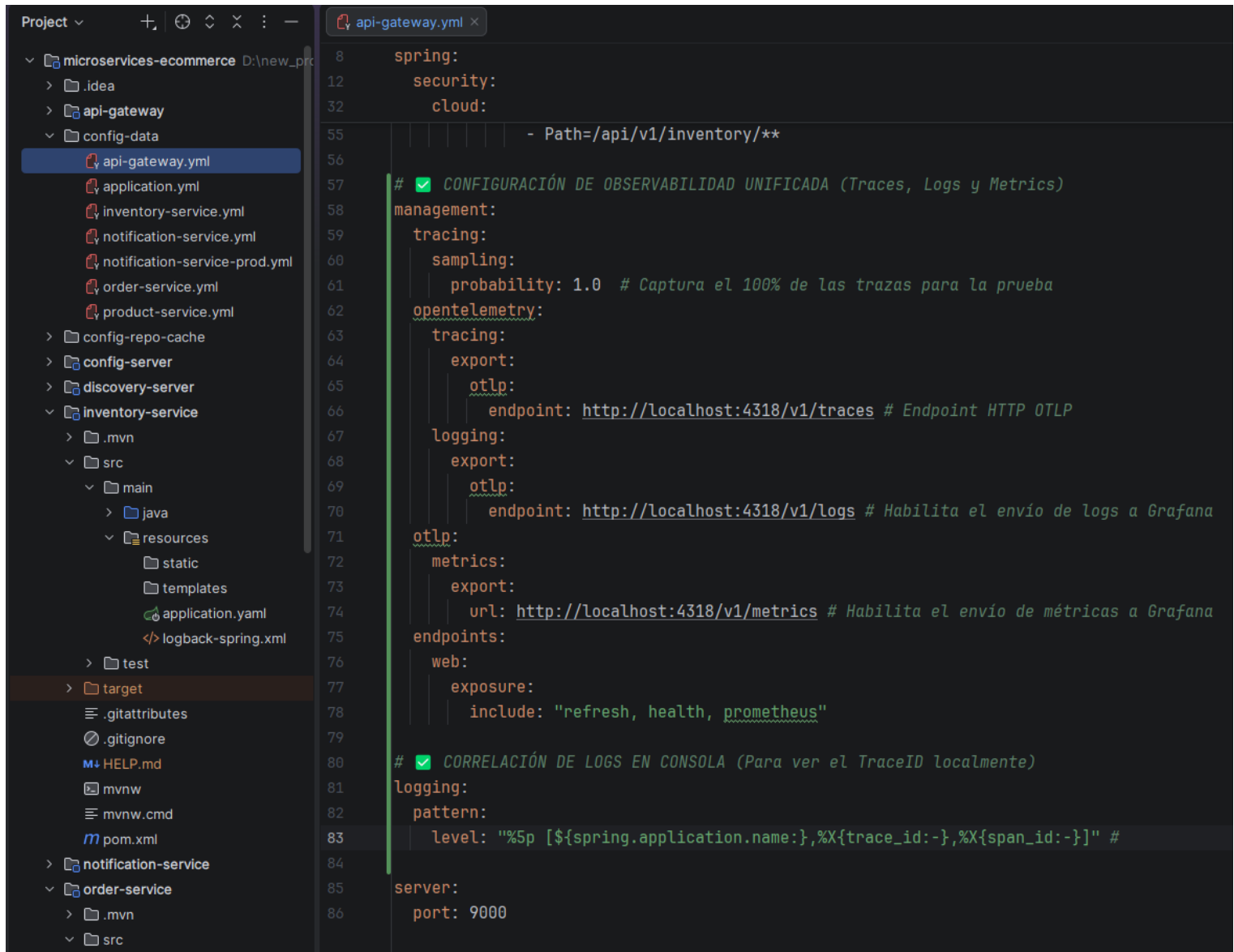
➔Config-data



```
23 client:
24     service-url:
25         defaultZone: http://localhost:8761/eureka/
26
27 # Por defecto, estará inhabilitado
28 inventory:
29     allow-backorders: false
30
31 management:
32     tracing:
33         sampling:
34             probability: 1.0
35         opentelemetry:
36             tracing:
37                 export:
38                     otlp:
39                         endpoint: http://localhost:4318/v1/traces
40             logging:
41                 export:
42                     otlp:
43                         endpoint: http://localhost:4318/v1/logs
44         otlp:
45             metrics:
46                 export:
47                     url: http://localhost:4318/v1/metrics
48         endpoints:
49             web:
50                 exposure:
51                     # Agregamos prometheus para que el stack LGTM pueda leer las métricas
52                     include: "refresh, health, prometheus"
53
54 # CORRELACIÓN DE LOGS (Para que el inventario reporte IDs en la consola)
55 logging:
56     pattern:
57         level: "%5p [%${spring.application.name:},%X{trace_id:-},%X{span_id:-}]"
```

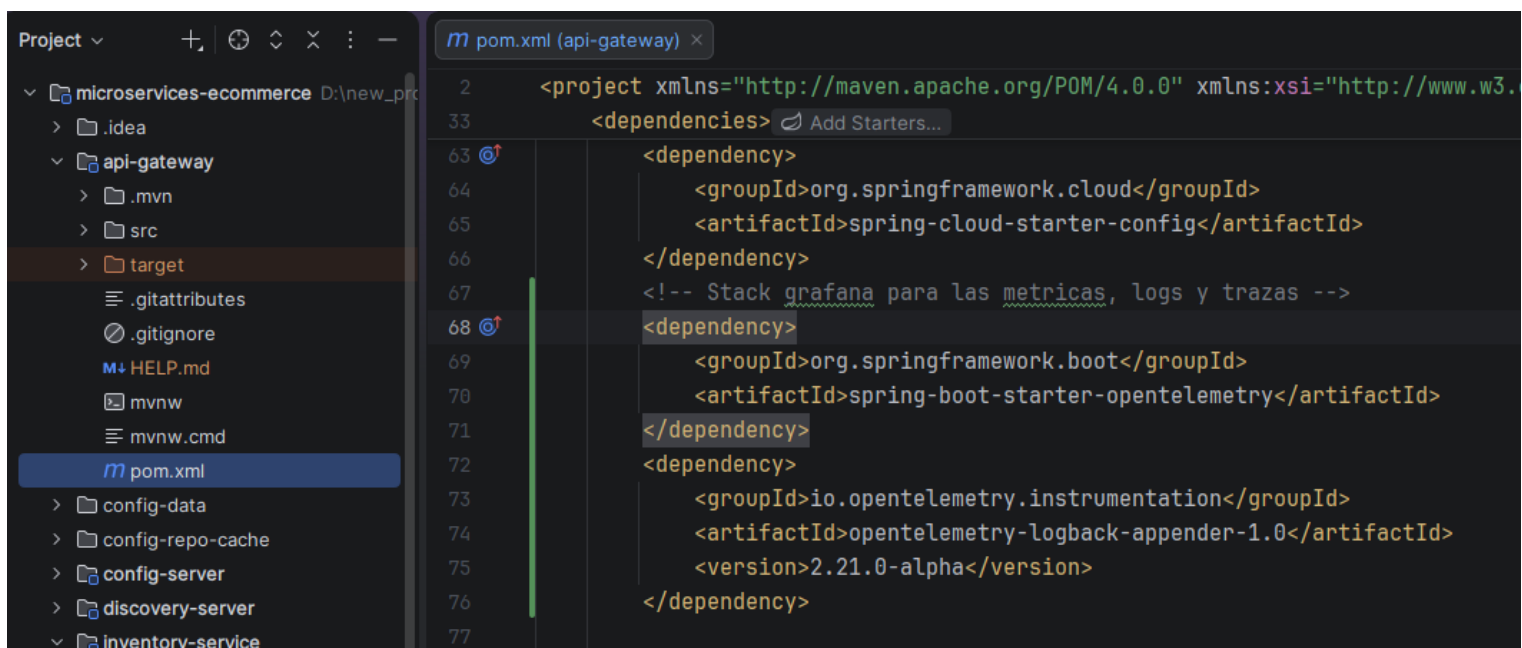
➔ Replicar en api-gateway:

<https://gist.github.com/gchaldy/67b45a2d87eee6e1b116181fb447032e>



```
Project ▾
├── microservices-e-commerce D:\new_pro...
│   ├── .idea
│   ├── api-gateway
│   └── config-data
│       ├── api-gateway.yml
│       ├── application.yml
│       ├── inventory-service.yml
│       ├── notification-service.yml
│       ├── notification-service-prod.yml
│       ├── order-service.yml
│       └── product-service.yml
├── config-repo-cache
├── config-server
├── discovery-server
├── inventory-service
│   ├── .mvn
│   └── src
│       ├── main
│       │   ├── java
│       │   └── resources
│       │       ├── static
│       │       ├── templates
│       │       ├── application.yml
│       │       └── logback-spring.xml
│       └── test
├── target
├── .gitattributes
├── .gitignore
├── HELP.md
├── mvnw
├── mvnw.cmd
├── pom.xml
├── notification-service
├── order-service
│   ├── .mvn
│   └── src
```

```
api-gateway.yml
8      spring:
12         security:
32         cloud:
55         - Path=/api/v1/inventory/**
56
57         # ✅ CONFIGURACIÓN DE OBSERVABILIDAD UNIFICADA (Traces, Logs y Metrics)
58         management:
59             tracing:
60                 sampling:
61                     probability: 1.0 # Captura el 100% de las trazas para la prueba
62             opentelemetry:
63                 tracing:
64                     export:
65                         otlp:
66                             endpoint: http://localhost:4318/v1/traces # Endpoint HTTP OTLP
67             logging:
68                 export:
69                     otlp:
70                         endpoint: http://localhost:4318/v1/logs # Habilita el envío de logs a Grafana
71             otlp:
72                 metrics:
73                     export:
74                         url: http://localhost:4318/v1/metrics # Habilita el envío de métricas a Grafana
75             endpoints:
76                 web:
77                     exposure:
78                         include: "refresh, health, prometheus"
79
80         # ✅ CORRELACIÓN DE LOGS EN CONSOLA (Para ver el TraceID localmente)
81         logging:
82             pattern:
83                 level: "%5p [%${spring.application.name:},%X{trace_id:-},%X{span_id:-}]" #
84
85         server:
86             port: 9000
```



```
Project ▾
├── microservices-e-commerce D:\new_pro...
│   ├── .idea
│   ├── api-gateway
│   │   ├── .mvn
│   │   └── src
│   │       ├── target
│   │       ├── .gitattributes
│   │       ├── .gitignore
│   │       ├── HELP.md
│   │       ├── mvnw
│   │       ├── mvnw.cmd
│   │       └── pom.xml
│   ├── config-data
│   ├── config-repo-cache
│   ├── config-server
│   ├── discovery-server
│   └── inventory-service
```

```
m pom.xml (api-gateway)
2      <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.
33      <dependencies> Add Starters...
63      <dependency>
64          <groupId>org.springframework.cloud</groupId>
65          <artifactId>spring-cloud-starter-config</artifactId>
66      </dependency>
67      <!-- Stack grafana para las metricas, logs y trazas -->
68      <dependency>
69          <groupId>org.springframework.boot</groupId>
70          <artifactId>spring-boot-starter-opentelemetry</artifactId>
71      </dependency>
72      <dependency>
73          <groupId>io.opentelemetry.instrumentation</groupId>
74          <artifactId>opentelemetry-logback-appender-1.0</artifactId>
75          <version>2.21.0-alpha</version>
76      </dependency>
77
```

Project ▾
microservices-e-commerce
 .idea
 api-gateway
 .mvn
 src
 main
 java
 com.ecommerce.api_gateway
 config
 GatewayConfig
 InstallOpenTelemetryAppender.java
 SecurityConfig
 enums
 ApiGatewayApplication

logback-spring.xml

InstallOpenTelemetryAppender.java

m pom.xml (api-gateway)

```
1 package com.ecommerce.api_gateway.config;
2
3 > import ...
4
5
6
7
8 @Component
9 class InstallOpenTelemetryAppender implements InitializingBean {
10     private final OpenTelemetry openTelemetry; 2 usages
11
12     InstallOpenTelemetryAppender(OpenTelemetry openTelemetry) { this.openTelemetry = openTelemetry; }
13
14
15
16     @Override no usages
17     public void afterPropertiesSet() { OpenTelemetryAppender.install(this.openTelemetry); }
18
19
20
21 }
```

Project ▾
microservices-e-commerce
 .idea
 api-gateway
 .mvn
 src
 main
 java
 com.ecommerce.api_gateway
 config
 GatewayConfig
 InstallOpenTelemetryAppender.java
 SecurityConfig
 enums
 ApiGatewayApplication
 resources
 application.yaml
 logback-spring.xml

logback-spring.xml

InstallOpenTelemetryAppender.java

m pom.xml (api-gateway)

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <configuration>
3     <include resource="org/springframework/boot/logging/logback/base.xml"/>
4     <!-- Procesador, entre los logs de springboot y opentelemetry -->
5     <appender name="OTEL" class="io.opentelemetry.instrumentation.logback.appender.v1_0.OpenTelemetryAppender"/>
6     <root level="INFO">
7         <appender-ref ref="CONSOLE"/> <!--por default en springboot, manda el mensaje directo a intelliJ-->
8         <appender-ref ref="OTEL"/> <!-- personal -->
9     </root>
10 </configuration>
```

163. Diagnóstico y Monitoreo de Microservicios

Query type: Search | TraceQL | Service Graph | Import trace

Service Name: api-gateway x

Span Name: http get x | http post x

Status: Select value

Duration: span | > | e.g. 100ms, 1.2s | < | e.g. 100ms, 1.2s

Tags: span | Select tag | = | Select value

{resource.service.name="api-gateway" && (name="http get" || name="http post")}

Edit in TraceQL

Search | Limit: 20 | Spans: 3 | Table: Traces | Streaming: Disabled | Metrics: Options | Step: auto | Type: Range | Streaming: Disabled

+ Add query | Query inspector

Trace ID	Start time	Service	Name
a24046dfcaf09...	2026-02-26 09:14:39	api-gateway	http post
d357efd9d06be7c4d1d5eef45d61f4d	2026-02-26 09:13:28	api-gateway	http get
484c676d926b...	2026-02-26 09:13:23	api-gateway	http get

Trace ID: d357efd9d06be7c4d1d5eef45d61f4d | Start time: 2026-02-26 09:13:28.024 (26 minutes ago) | Duration: 381.19ms | Services: 2 | URL: /api/v1/order

Node graph

Trace: api-gateway: http get | Feedback

Span Filters: 12 spans

Service & Operation graph

api-gateway http get (381.19ms)

security filterchain before (11.7ms)

authorize exchange (8.03ms)

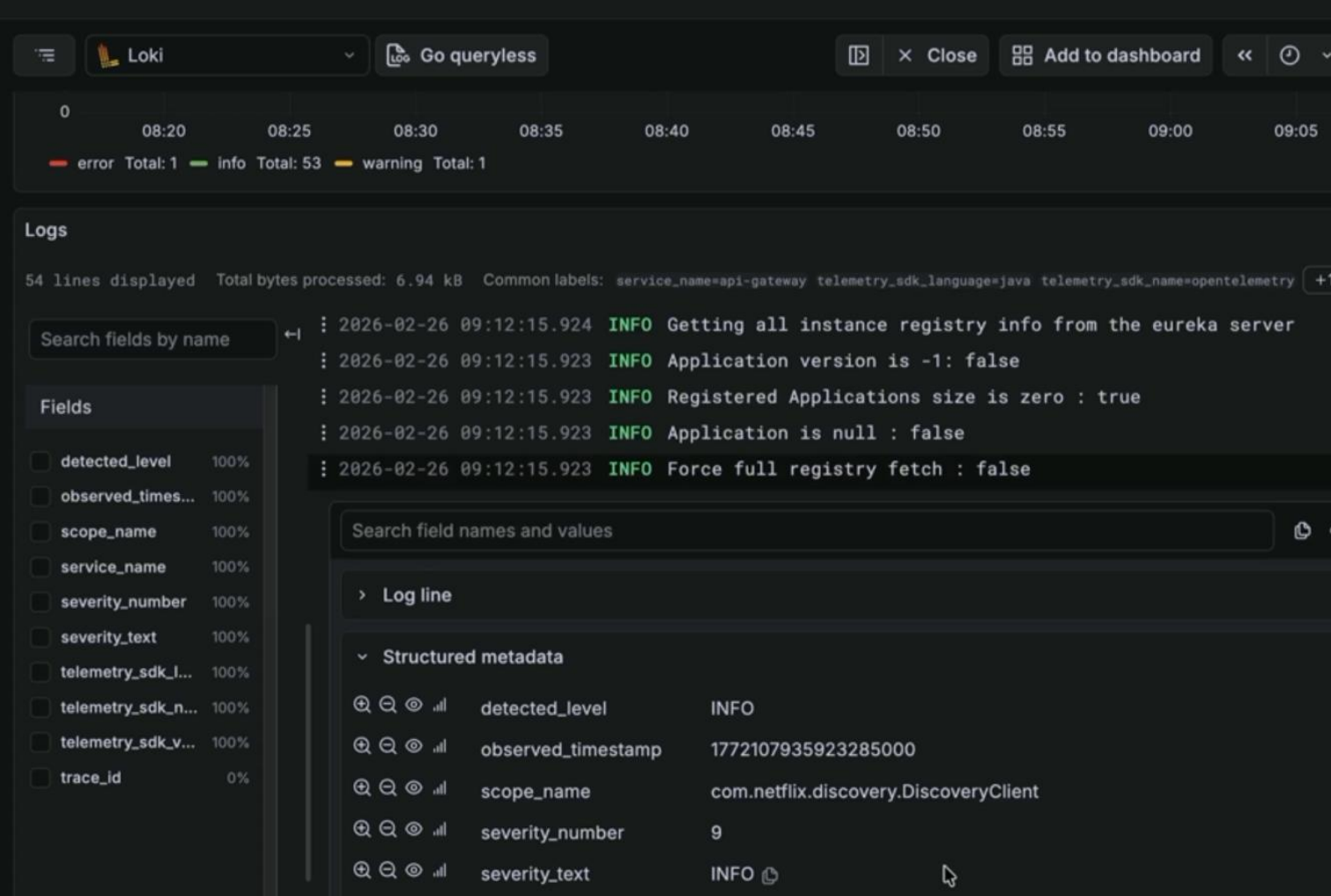
secured request (368.5ms)

HTTP GET (341.01ms)

order-service http get /api/v1/

security filterchain before (8.03ms)

authenticate bearer token (89.77ms)



164. Prometheus: Las métricas

